



PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS

Instituto de Ciências Exatas e de Informática

Relatório Atividade Prática #2 – Algoritmos de Busca no Jogo 8-Puzzle*

Bernardo Ladeira Kartabil¹

Guilherme Henrique da Silva Teodoro²

Marcella Santos Belchior³

Yasmin Torres Moreira dos Santos⁴

Resumo

Este relatório documenta a modelagem formal e a implementação computacional de algoritmos de busca para a resolução do clássico problema do 8-Puzzle. O trabalho tem como foco a análise e a comparação de desempenho empírico entre uma suíte abrangente de abordagens de busca cega (Busca em Largura, Busca em Profundidade e Custo Uniforme) e busca informada (Busca Gulosa e A*). Para orientar as buscas heurísticas, foram concebidas e analisadas três funções de estimativa de custo: contagem de peças fora do lugar, distância de Manhattan e uma extensão considerando penalidades por conflito linear. Os resultados obtidos em baterias de testes com diferentes níveis de complexidade permitem contrastar as características de completude e otimalidade de cada método. A análise evidencia o impacto das estratégias na redução do espaço de estados e ilustra os variados trade-offs computacionais entre consumo de memória, tempo de execução e eficiência na exploração da árvore de busca.

Palavras-chave: Inteligência Artificial. Busca Cega. Busca Heurística. 8-Puzzle. Algoritmos de Busca.

*Relatório apresentado ao Instituto de Ciências Exatas e Informática da Pontifícia Universidade Católica de Minas Gerais como requisito da disciplina de Inteligência Artificial.

¹Graduando em Ciência da Computação, Brasil – bernardo.kartabil@sga.pucminas.br.

²Graduando em Ciência da Computação, Brasil – ghsteodoro@sga.pucminas.br.

³Graduanda em Ciência da Computação, Brasil – marcella.belchior@sga.pucminas.br.

⁴Graduanda em Ciência da Computação, Brasil – yasmin.santos.1484596@sga.pucminas.br.

1 INTRODUÇÃO E FORMALIZAÇÃO DO PROBLEMA

O 8-Puzzle é modelado como um problema de busca em espaço de estados. Um estado é uma tupla imutável de nove inteiros, em ordem de leitura do tabuleiro 3×3 ; os valores de 1 a 8 identificam as peças e 0 representa o espaço vazio (na implementação, o zero é representado por "_"). O estado inicial é qualquer configuração válida informada pelo usuário ou selecionada nos casos de teste. O estado objetivo adotado é (1, 2, 3, 4, 5, 6, 7, 8, 0).

Para tal, a literatura dispõe de algoritmos de busca cega (como Busca em Largura, Profundidade e Custo Uniforme), que exploram a árvore de estados de maneira sistemática e exaustiva, bem como algoritmos de busca informada (como A-Estrela e Gulosa), que utilizam aproximações heurísticas para guiar o agente em direção à solução com maior eficiência.

O ambiente de execução do 8-Puzzle, comum a todos os algoritmos avaliados neste trabalho, possui características bem definidas:

- **Totalmente observável:** O estado completo do tabuleiro e a posição de cada peça são conhecidos a cada instante.
- **Determinístico:** O resultado da transição de um estado após aplicar uma ação (mover o espaço vazio) é estritamente previsível.
- **Estático:** O tabuleiro e suas regras não sofrem qualquer alteração externa enquanto o algoritmo processa e delibera sobre os caminhos.
- **Discreto:** Existe um conjunto restrito, enumerável e finito de configurações possíveis (estados) e um número máximo de 4 movimentos (ações).

2 MODELAGEM DA BUSCA

Para que os algoritmos de busca (tanto cegos quanto informados) possam operar de maneira padronizada e modular, o quebra-cabeça foi formulado computacionalmente como um problema de espaço de estados. A arquitetura foi construída de modo que o motor de regras seja completamente desacoplado da lógica de travessia dos algoritmos.

A formulação explícita do problema segue os seguintes componentes:

- **Representação do Estado:** Cada configuração do tabuleiro é encapsulada em um objeto, onde a disposição das peças é representada por uma tupla unidimensional de 9 números inteiros. O valor 0 (zero) é utilizado para denotar o espaço vazio. Essa estrutura de tupla garante imutabilidade e permite a geração eficiente de hashes (identificadores únicos) para o controle de estados já explorados em conjuntos (*sets*) de memória.
- **Estado Inicial:** Qualquer permutação válida da tupla que possua solução matemática. A validade é verificada no pré-processamento através da contagem de inversões (pares

de peças fora de ordem), onde um número ímpar de inversões indica um tabuleiro sem solução.

- **Teste de Objetivo:** Uma verificação booleana de igualdade que compara a tupla do estado atual com a tupla da configuração alvo: (1, 2, 3, 4, 5, 6, 7, 8, 0).
- **Conjunto de Ações:** Consiste nos movimentos ortogonais possíveis do espaço vazio em relação aos limites da matriz geométrica 3×3 . As ações permitidas são: *Cima*, *Baixo*, *Esquerda* e *Direita*.
- **Modelo de Transição:** Uma função responsável por receber um estado pai e gerar a lista de nós filhos válidos (sucessores). O modelo calcula a troca de posições (índices) no vetor unidimensional simulando a grade bidimensional, descartando ações que fariam o espaço vazio ultrapassar as bordas do tabuleiro.
- **Custo do Caminho:** O custo transicional de mover uma peça é sempre uniforme. Logo, o custo do passo é $c = 1$. Consequentemente, o custo total do caminho percorrido até um nó n , denotado por $g(n)$, é estritamente igual à profundidade daquele nó na árvore de busca.

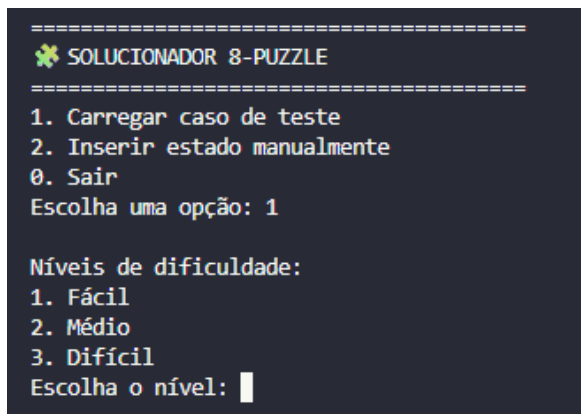


Figura 1 – Tela de inserção da dificuldade do estado inicial.

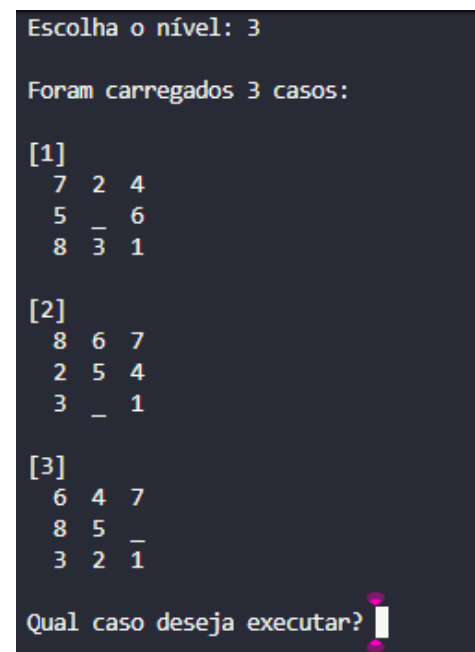


Figura 2 – Seleção de opções do estado de acordo com a dificuldade escolhida .

```

Qual caso deseja executar? 1

-----
Estado Selecionado:

  7  2  4
  5  _  6
  8  3  1

-----

1. A* (A-Estrela)
2. Busca em Largura (BFS)
3. Busca em Profundidade (DFS)
4. Busca de Custo Uniforme
5. Busca Gulosa
0. Voltar
Selecione o algoritmo: 1

-----

Heurísticas disponíveis para o A*:
1. Peças fora do lugar
2. Distância de Manhattan
3. Conflito Linear
0. Voltar
Selecione a heurística:

```

Figura 3 – Tela de seleção do algoritmo de busca.(A + heurística)

```

  7  8  _
-----

MÉTRICAS DA EXECUÇÃO:
- Profundidade da solução: 1 movimentos
- Número de nós visitados: 2
- Número de nós gerados: 4
- Tempo de execução: 0.00008 segundos
=====

```

Figura 4 – Exemplo de retorno do algoritmo escolhido (A* + heurística)

3 ALGORITMOS E HEURÍSTICAS IMPLEMENTADOS

3.1 Algoritmos de Busca

Para a resolução computacional, foram implementados algoritmos baseados em árvores de busca, estruturados através do encapsulamento de nós. Para abranger tanto a exploração exaustiva quanto a exploração guiada, a suíte de algoritmos contempla cinco abordagens distintas.

1. Busca em Largura (BFS - Breadth-First Search)

Funcionamento: A BFS explora a árvore de busca nível por nível, expandindo sempre o nó mais raso da fronteira. A implementação utiliza uma fila padrão (FIFO - *First-In, First-Out*).

Compleitude e Otimalidade: É um algoritmo completo em espaços de estados finitos. No contexto do 8-Puzzle, onde todos os custos de passo são uniformes ($c = 1$), a BFS garante a otimalidade, encontrando sempre o caminho com o menor número de movimentos.

Complexidade: Suas complexidades de tempo e espaço são $O(b^d)$, onde b é o fator de ramificação (média de movimentos possíveis, ≈ 3 no 8-Puzzle) e d é a profundidade da solução mais rasa. O consumo de memória é seu maior gargalo lógico.

2. Busca em Profundidade (DFS - Depth-First Search)

Funcionamento: A DFS explora primeiro o sucessor mais profundo disponível. A implementação usa uma pilha (LIFO - *Last-In, First-Out*): remove o topo, verifica se é objetivo e, caso não seja, insere seus sucessores. Embora os sucessores sejam obtidos na ordem Cima, Baixo, Esquerda e Direita, eles são empilhados na ordem inversa para preservar a prioridade na expansão. O conjunto de estados explorados evita ciclos.

Completeness e Otimalidade: No espaço finito do 8-Puzzle, com controle de estados repetidos, a DFS é completa. No entanto, não é ótima, pois o primeiro caminho encontrado pode ser extremamente longo em comparação à solução ideal.

Complexidade: As complexidades de pior caso são $O(b^m)$ em tempo e espaço, em que m é a profundidade máxima explorada.

3. Busca de Custo Uniforme (UCS)

Funcionamento: Expande o nó com o menor custo de caminho acumulado, $g(n)$. Utiliza uma fila de prioridade ordenada por esse custo.

Completeness e Otimalidade: Como todos os custos de passo são estritamente positivos, a UCS é completa e ótima.

Complexidade: No 8-Puzzle, como o custo de passo é sempre 1, a UCS comporta-se de maneira idêntica à Busca em Largura, explorando em anéis concêntricos. Sua complexidade de tempo e espaço é $O(b^{1+\lceil C^*/\epsilon \rceil})$, onde C^* é o custo da solução ótima, o que neste domínio se simplifica para $O(b^d)$.

4. Busca Gulosa (Greedy Search)

Funcionamento: É uma busca informada que ordena a fronteira apenas pelo valor heurístico $h(n)$, usando uma fila de prioridade. A cada passo, seleciona o estado que aparenta estar mais próximo do objetivo, ignorando o custo já percorrido $g(n)$. Na implementação, o algoritmo utiliza a Distância de Manhattan como função heurística padrão para a avaliação dos sucessores.

Completeness e Otimalidade: No 8-Puzzle finito, com controle de estados repetidos, ela é completa. Porém, não garante uma solução ótima, pois a ansiedade da heurística pode preferir um estado promissor que, na verdade, exige um caminho longo e custoso para ser resolvido.

Complexidade: O pior caso continua exponencial, $O(b^m)$ em tempo e espaço, embora o desempenho prático dependa diretamente da qualidade da heurística para podar caminhos ruins.

5. Algoritmo A-Estrela (A^*)

Funcionamento: Combina as vantagens da UCS e da Busca Gulosa, ordenando a fila de prioridade pela função de avaliação $f(n) = g(n) + h(n)$. Ele pondera o custo já percorrido com a estimativa de custo até o objetivo.

Completo e Otimalidade: É completo e garante otimalidade sob uma condição matemática restrita: a heurística $h(n)$ deve ser *admissível* (nunca superestimar o custo real).

Complexidade: Embora a complexidade de tempo dependa fortemente da heurística, o A^* é otimamente eficiente (nenhum outro algoritmo expandirá menos nós com a mesma heurística). A complexidade de espaço ainda é $O(b^d)$, exigindo a manutenção de todos os nós gerados em memória.

3.2 Heurísticas Utilizadas e Admissibilidade

A eficiência das buscas informadas (A^* e Gulosa) depende da formulação das heurísticas. Para garantir a otimalidade do A^* , todas as heurísticas propostas neste projeto são admissíveis, ou seja, $h(n) \leq h^*(n)$, onde $h^*(n)$ é o custo real para atingir o objetivo.

1. **Peças fora do lugar (Distância de Hamming):** Contabiliza a quantidade exata de blocos que não se encontram em suas posições finais (ignorando o espaço vazio).

Admissibilidade: É admissível porque qualquer peça fora do lugar precisará ser movida no mínimo uma vez para chegar ao seu destino.

2. **Distância de Manhattan:** Calcula a soma dos deslocamentos absolutos nos eixos vertical e horizontal que cada peça precisa percorrer até o objetivo, ignorando os bloqueios de outras peças.

Admissibilidade: É admissível porque uma peça não pode se mover diagonalmente. O número de quarteirões calculado por Manhattan é o número absoluto mínimo de passos matemáticos necessários para alinhar a peça.

3. **Conflito Linear:** Uma penalidade adicional somada à Distância de Manhattan. Aplica um custo extra de 2 movimentos sempre que duas peças estiverem na mesma linha ou coluna final, mas com suas posições invertidas, bloqueando-se mutuamente (uma precisará sair do caminho e voltar).

Admissibilidade: É admissível e mais informada (dominante) em relação à Manhattan. Se duas peças estão em conflito linear, é provado matematicamente que a distância de Manhattan subestimou o custo real em pelo menos dois movimentos. Logo, somar 2 ao custo mantém a heurística estritamente abaixo ou igual ao custo real.

4 RESULTADOS EXPERIMENTAIS

Os testes foram executados utilizando estados iniciais categorizados por nível de dificuldade.

Tabela 1 – Resultados do Algoritmo A*

Dificuldade	Heurística	Profund.	Nós Visit.	Nós Gerados	Tempo (s)
Fácil (Caso 1)	1. Peças fora do lugar	1	2	4	0,00012
	2. Dist. Manhattan	1	2	4	0,00130
	3. Conflito Linear	1	2	4	0,00059
Fácil (Caso 2)	1. Peças fora do lugar	2	3	5	0,00016
	2. Dist. Manhattan	2	3	5	0,00011
	3. Conflito Linear	2	3	5	0,00020
Fácil (Caso 3)	1. Peças fora do lugar	1	2	4	0,00007
	2. Dist. Manhattan	1	2	4	0,00008
	3. Conflito Linear	1	2	4	0,00034
Médio (Caso 1)	1. Peças fora do lugar	4	5	10	0,00013
	2. Dist. Manhattan	4	5	10	0,00015
	3. Conflito Linear	4	5	10	0,00031
Médio (Caso 2)	1. Peças fora do lugar	7	8	15	0,00012
	2. Dist. Manhattan	7	8	15	0,00019
	3. Conflito Linear	7	8	15	0,00045
Médio (Caso 3)	1. Peças fora do lugar	8	19	34	0,00030
	2. Dist. Manhattan	8	13	24	0,00025
	3. Conflito Linear	8	13	24	0,00080
Difícil (Caso 1)	1. Peças fora do lugar	20	3654	6025	0,05872
	2. Dist. Manhattan	20	271	443	0,00548
	3. Conflito Linear	20	150	252	0,00420
Difícil (Caso 2)	1. Peças fora do lugar	31	122287	179139	2,02819
	2. Dist. Manhattan	31	7397	11619	0,11589
	3. Conflito Linear	31	5358	8762	0,15553
Difícil (Caso 3)	1. Peças fora do lugar	31	121681	178447	1,87953
	2. Dist. Manhattan	31	10603	17002	0,18082
	3. Conflito Linear	31	5488	8976	0,18233

Tabela 2 – Resultados da Busca Gulosa (Manhattan)

Dificuldade	Heurística	Profund.	Nós Visit.	Nós Ger.	Tempo (s)
Fácil (Caso 1)	Manhattan	1	2	4	0,00009
Fácil (Caso 2)	Manhattan	2	3	5	0,00011
Fácil (Caso 3)	Manhattan	1	2	4	0,00007
Médio (Caso 1)	Manhattan	4	5	10	0,00012
Médio (Caso 2)	Manhattan	7	8	15	0,00016
Médio (Caso 3)	Manhattan	10	25	47	0,00056
Difícil (Caso 1)	Manhattan	62	548	909	0,01226
Difícil (Caso 2)	Manhattan	47	100	161	0,00217
Difícil (Caso 3)	Manhattan	47	123	202	0,00217

Tabela 3 – Resultados da Busca em Largura (BFS)

Dificuldade	Profund.	Nós Visit.	Nós Ger.	Tempo (s)
Fácil (Caso 1)	1	4	8	0,00321
Fácil (Caso 2)	2	7	14	0,00012
Fácil (Caso 3)	1	3	5	0,00008
Médio (Caso 1)	4	33	60	0,00028
Médio (Caso 2)	7	111	198	0,00091
Médio (Caso 3)	8	311	500	0,00211
Difícil (Caso 1)	20	55409	74203	0,36491
Difícil (Caso 2)	31	181439	181440	1,16557
Difícil (Caso 3)	31	181440	181440	1,18237

Tabela 4 – Resultados da Busca em Profundidade (DFS)

Dificuldade	Profundidade	Nós Visit.	Nós Gerados	Tempo (s)
Fácil (Caso 1)	1	181440	181440	0,50192
Fácil (Caso 2)	64328	84897	127093	0,26060
Fácil (Caso 3)	1	181439	181440	0,64925
Médio (Caso 1)	54710	127975	165588	0,41362
Médio (Caso 2)	51551	58705	94625	0,25495
Médio (Caso 3)	52270	131502	167832	0,53366
Difícil (Caso 1)	39478	146987	175856	0,55241
Difícil (Caso 2)	40757	44186	73832	0,13817
Difícil (Caso 3)	18055	166721	180799	0,57164

Tabela 5 – Resultados da Busca de Custo Uniforme (UCS)

Dificuldade	Profundidade	Nós Visit.	Nós Ger.	Tempo (s)
Fácil (Caso 1)	1	4	8	0,00138
Fácil (Caso 2)	2	7	14	0,00027
Fácil (Caso 3)	1	3	5	0,00006
Médio (Caso 1)	4	33	60	0,00056
Médio (Caso 2)	7	111	198	0,00464
Médio (Caso 3)	8	311	500	0,00307
Difícil (Caso 1)	20	55409	74203	0,53130
Difícil (Caso 2)	31	181439	181440	2,03124
Difícil (Caso 3)	31	181440	181440	1,58028

5 ANÁLISE COMPARATIVA (ITEM F)

5.1 Desempenho e Consistência (a)

Os resultados demonstram que, entre os algoritmos de busca cega (BFS, DFS e UCS), a BFS e a UCS comportam-se de forma idêntica no 8-Puzzle devido ao custo uniforme ($c = 1$), sendo ótimas mas computacionalmente caras para casos difíceis. A DFS, embora rápida em alguns casos, revelou-se instável e não ótima, gerando profundidades de solução inconsistentes. O algoritmo mais rápido e eficiente foi o A* com a heurística de Conflito Linear, que visitou ordens de magnitude menos nós que os algoritmos cegos, sendo consistente em todas as instâncias experimentais.

5.2 Impacto das Heurísticas no A* (b)

As heurísticas impactaram drasticamente a eficiência do A*.

- **Peças fora do lugar:** Foi a menos eficiente, fornecendo uma estimativa menos precisa e exigindo a expansão de mais nós.
- **Distância de Manhattan:** Apresentou desempenho superior, fornecendo uma estimativa de custo mais próxima da realidade (admissível e consistente).
- **Conflito Linear:** Foi a mais eficiente de todas. Ao identificar que peças na linha ou coluna correta podem estar bloqueadas, ela adiciona uma penalidade matemática que "poda" caminhos ruins, reduzindo significativamente o número de nós gerados e visitados em relação à Manhattan simples.

5.3 Diferença nos Caminhos Encontrados (c)

Sim, houve diferenças. Algoritmos ótimos (BFS, UCS e A*) encontraram caminhos com o mesmo custo mínimo de passos. Contudo, a DFS e a Busca Gulosa apresentaram soluções com profundidades superiores. A DFS falha em garantir otimalidade por focar apenas na profundidade, enquanto a Gulosa, guiada cegamente pela heurística, pode seguir um "caminho atrativo" que na realidade é mais longo. A diferença reside na estratégia de escolha: enquanto algoritmos ótimos equilibram custo real $g(n)$ e estimativa $h(n)$, os não ótimos ignoram um desses componentes.

5.4 Recomendações de Uso (d)

1. **A* com Conflito Linear:** Recomendado para quase todas as situações, especialmente em problemas complexos onde a otimalidade e o tempo de resposta são cruciais.
2. **Busca Gulosa:** Recomendada apenas em cenários onde o espaço de estados é gigantesco e uma solução "boa o suficiente" é necessária rapidamente, aceitando que ela pode não ser a melhor (ótima).
3. **Busca em Largura/UCS:** Recomendadas apenas para estados com profundidade de solução muito curta, onde a garantia absoluta de otimalidade é necessária e a memória não é uma restrição severa.
4. **Busca em Profundidade:** Não recomendada para o 8-Puzzle, exceto em casos onde a memória é extremamente limitada e a solução está garantidamente próxima ao estado inicial.

6 DIÁRIO DE DECISÕES (ITEM G)

a. Maior dificuldade técnica e resolução

A maior dificuldade técnica residiu na modelagem da heurística de *Conflito Linear*. Inicialmente, a implementação apenas da Distância de Manhattan apresentava resultados satisfatórios, mas a transposição teórica do conceito de conflito linear para a prática computacional exigiu uma análise cuidadosa dos índices bidimensionais dentro de uma estrutura unidimensional. A solução foi o desenvolvimento de uma função auxiliar que mapeia os conflitos de linha e coluna através de um dicionário de posições alvo, garantindo que a complexidade algorítmica não fosse prejudicada pela verificação redundante de estados.

b. O que foi tentado e não funcionou

Durante o desenvolvimento, tentamos implementar a Busca em Profundidade Iterativa (IDDFS) com o intuito de unir a economia de memória da DFS com a otimalidade da BFS. Contudo, devido à natureza específica do 8-Puzzle, que possui muitos caminhos redundantes, a re-expansão constante de estados em cada incremento da profundidade tornou a execução excessivamente lenta em comparação ao A*. Outra tentativa frustrada foi o uso de listas simples para a *fronteira* do A*, o que resultou em um gargalo de desempenho significativo. A substituição por filas de prioridade (*heapq*) foi a correção necessária para atingir a eficiência atual.

c. Melhorias e explorações futuras

Se houvesse maior disponibilidade de tempo, a prioridade seria a transição da interface atual, baseada em terminal, para uma interface gráfica (GUI) robusta e centrada em experiência do usuário (UI/UX). Uma interface visual permitiria o acompanhamento em tempo real da "árvore de busca" e da movimentação das peças, facilitando a interpretação do comportamento da IA para usuários leigos. Além disso, exploraríamos a implementação do algoritmo IDA* (Iterative Deepening A*), que combina a eficácia heurística do A* com o baixo consumo de memória da busca em profundidade, sendo a fronteira lógica para a resolução de *puzzles* de dimensões superiores a 3×3 .